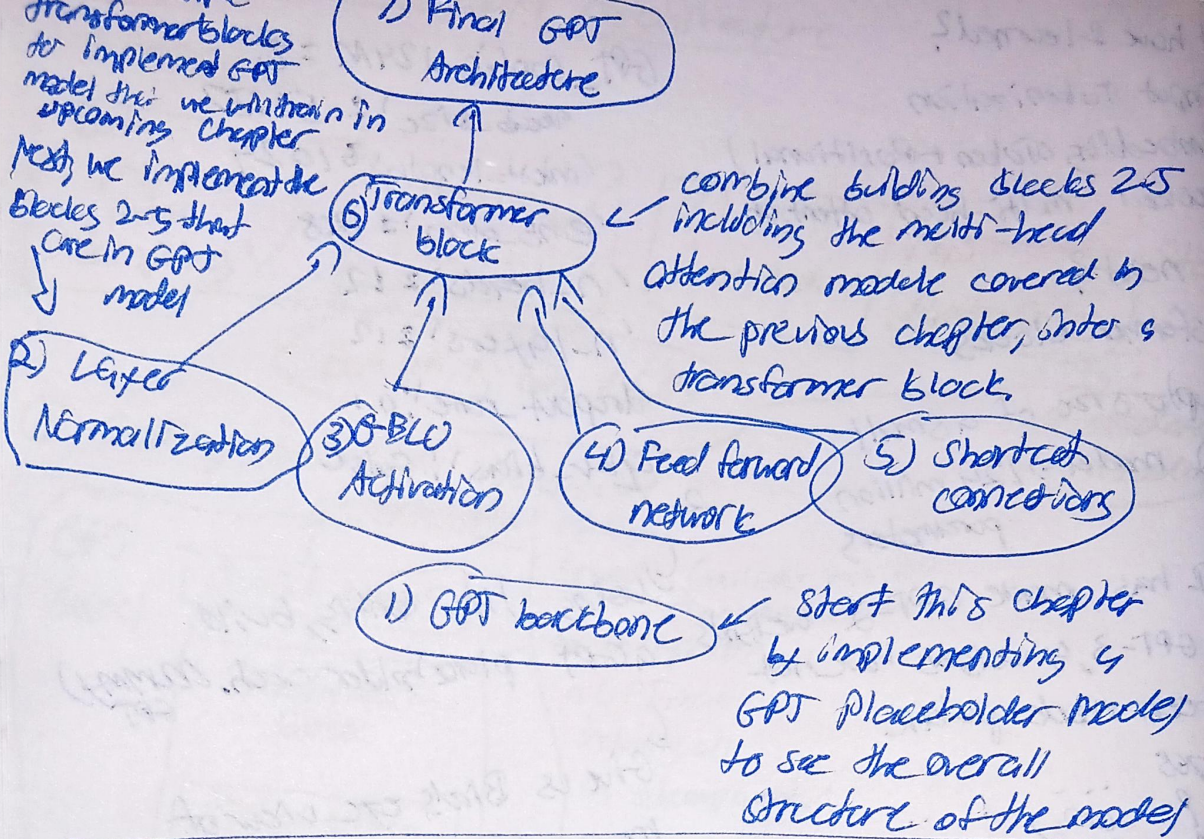




Layer Normalization

MN = Neural Network

- ① Training deep MNs w/ many layers can be challenging due to vanishing/exploding gradients
- ② This leads to unstable training dynamics
- ③ Layer normalization improves stability and efficiency of MN training
- ④ Main idea: Adjust outputs of MN to have mean zero and variance of 1 → Speeds up convergence

Gradient

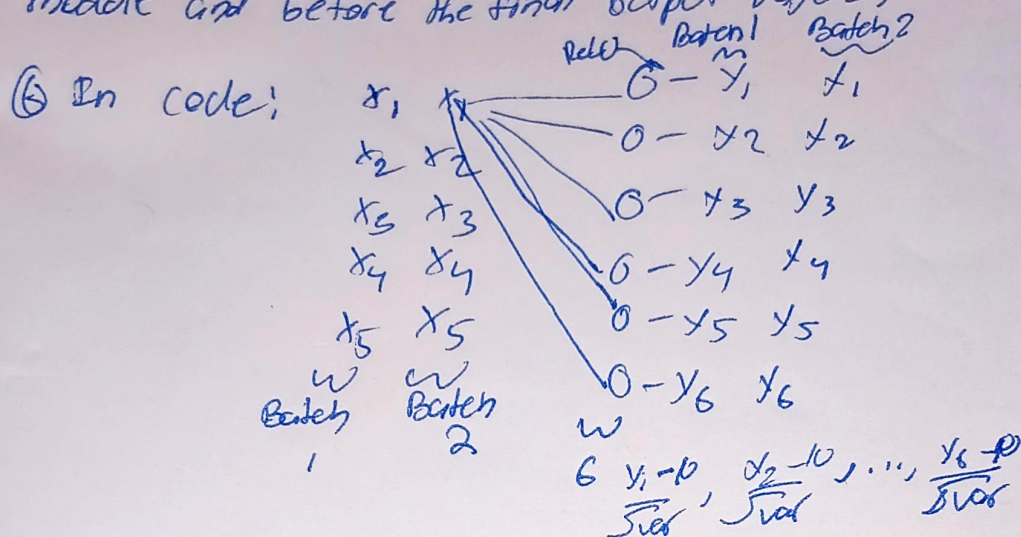
- Depends on layer output
- If layer output is too large or too small, gradient magnitude can become too large or too small
- As training proceeds, inputs to each layer can change (neural network shift)
- This delays convergence
- Layer normalization prevents this
- Affects training
- Layer Batch normalization keeps gradients stable

Ex: $\begin{matrix} x_1 & x_2 & x_3 & x_4 \\ [11, 0.8, 2.3, 4.4] \end{matrix}$ $\text{mean} = \mu = \frac{x_1 + x_2 + x_3 + x_4}{4} = 2.15$

$$\text{var} = \text{var} = \frac{1}{4} [(x_1 - \mu)^2 + (x_2 - \mu)^2 + (x_3 - \mu)^2 + (x_4 - \mu)^2] = 2.0025$$

$$\text{normalized} = \left[\frac{x_1 - \mu}{\sqrt{\text{var}}}, \frac{x_2 - \mu}{\sqrt{\text{var}}}, \frac{x_3 - \mu}{\sqrt{\text{var}}}, \frac{x_4 - \mu}{\sqrt{\text{var}}} \right] = [-0.74, -0.95, 0.1, 1.59]$$

⑤ In GPT-2, and modern transformer architectures, Layer normalization is typically applied before and after the multi-head attention module and before the final output layers, $\mu = \text{mean}$



⑦ Layer vs Batch normalization

Normalize along feature dimension, independently of batch size

Available hardware
Opticaly
dictates
batch size

↓
Leads to more flexibility and stability for distributed training/environments which lack resources